2026년도 전국기능경기대회 채점기준

1. 채점상의 유의사항	직 종 명	클라우드컴퓨팅

※ 다음 사항을 유의하여 채점하시오.

1) AWS의 지역은 ap-northeast-2을 사용합니다.
2) 웹페이지 접근은 크롬이나 파이어폭스를 이용합니다.
3) 웹페이지에서 언어에 따라 문구가 다르게 보일 수 있습니다.
4) shell에서의 명령어의 출력은 버전에 따라 조금 다를 수 있습니다.
5) 문제지와 채점지에 있는 <> 는 변수입니다. 해당 부분을 변경해 입력합니다.
6) 채점은 문항 순서대로 진행해야 합니다.
7) 삭제된 채점자료는 되돌릴 수 없음으로 유의하여 진행하며, 이의신청까지 완료 이후 선수가 생성한 클라우드 리소스를 삭제합니다.
8) 부분 점수가 있는 문항은 채점 항목에 부분 점수가 적혀져 있습니다.
9) 부분 점수가 따로 없는 문항은 모두 맞아야 점수로 인정됩니다.
10) 리소스의 정보를 읽어오는 채점항목은 기본적으로 스크립트 결과를 통해 채점을 진행하며, 만약 선수가 이의가 있다면 명령어를 직접 입력하여 확인해볼 수 있습니다.
11) (예상 출력)은 바로 이전 (명령어 입력)의 예상 출력을 의미합니다.
12) 채점 시에는 별도로 제공한 채점 스크립트(mark.sh)를 실행하여 채점할 수 있습니다. 다만, 선수가 직접 입력을 원할 경우 채점기준표에 명시된 명령어 그대로 입력하여 채점할 수 있습니다.
13) 배포된 채점 스크립트(mark.sh) 는 ec2-user에 최상위 경로에 위치 하도록 합니다.
14) 모든 채점 사항은 wsc-bastion에서 ssh 접속 후 진행합니다.₩
15) EKS Node Group 접근을 위하여 Bastion을 통하여 접근합니다.

2. 채점기준표

1) 주요항목별 배점	직 종 명	클라우드컴퓨팅

과제 번호	일련 번호	주요항목	배점	채점방법		채점시기		비고
				독립	합의	경기 진행중	경기 종료후	
제1과제	1	VPC	1.5		○		○	
	2	DynamoDB	1.0		○		○	
	3	S3	1.0		○		○	
	4	Lambda	1.0		○		○	
	5	ECR	2.5		○		○	
	6	EKS	9.0		○		○	
	7	Load Balancer	1.5		○		○	
	8	CloudFront	2.0		○		○	
	9	WAF	1.0		○		○	
	10	App Service	5.5		○		○	
	11	Observability & Monitoring	2.5		○		○	
	12	Logging	1.5		○		○	
합 계			30					

2) 채점방법 및 기준

(경기종료 후 채점)

과제 번호	일련 번호	주요항목	일련 번호	세부항목(채점방법)	배점
1과제	1	VPC	1	Network Configure	1.5
	2	DynamoDB	1	DynamoDB Configure	1.0
	3	S3	1	S3 Configure	1.0
	4	Lambda	1	Lambda Configure	1.0
	5	ECR	1	ECR Configure	1.0
			2	ECR Image Configure	1.5
	6	EKS	1	EKS Cluster Configure	1.0
			2	EKS Node Group Configure	1.0
			3	EKS Node Group Storage Encrytion	1.5
			4	EKS Node Group Internet Test	1.5
			5	EKS Node Group IMDS Check	1.5
			6	EKS Cluster DNS Domain Check	1.5
			7	EKS Object Configure	1.0
	7	Load Balancer	1	Load Balancer Configure	1.5
	8	CloudFront	1	CloudFront Configure	1.0
			2	CloudFront Caching	1.0
	9	WAF	1	WAF Configure	1.0
	10	App Service	1	API POST Test	1.5
			2	API GET Test	1.5
			3	API Block Test	1.5
			4	Static Web Page Test	1.0
	11	Observability & Monitoring	1	Prometheus & Grafana Configure	1.0
			2	Grafana Dashboard Configure	1.5
	12	Logging	1	Logging Configure	1.5
	총점				30

3) 채점내용

<table>
<tr><th>순번</th><th colspan="2">채점항목</th></tr>
<tr><td></td><td colspan="2">1) SSH를 통해 EC2에 접근합니다.
2) 아래 파일을 EC2의 /home/ec2-user/marking/ 디렉터리로 복사합니다.
3) /home/ec2-user/marking/ 경로에서 스크립트를 실행합니다. 실행 결과를 기반으로 채점을 진행하되 선수가 이의를 제기할 경우 수동우로 채점을 진행할수 있도록 합니다.
4) 채점을 진행하기 전에 다음 명령어를 수행하여 채점 진행을 위한 사전 작업을 진행합니다.
5) 채점은 ap-northeast-2 region에 생성되어 있는 EC2에서 진행합니다.</td></tr>
<tr><td>0</td><td colspan="2">#mark.sh Set Command

set default region of aws cli
aws configure set default.region ap-northeast-2

set default output of aws cli
aws configure set default.output json</td></tr>
<tr><td rowspan="4">1-1</td><td>1-1-A
(명령어 입력)</td><td>aws ec2 describe-vpcs --filter Name=tag:Name,Values=wsc-vpc --query "Vpcs[].CidrBlock"
--output text</td></tr>
<tr><td>1-1-A
(예상 출력)
<u>정확히 일치</u></td><td>10.0.0.0/16</td></tr>
<tr><td>1-1-B
(명령어 입력)</td><td>aws ec2 describe-subnets --filters Name=tag:Name,Values=wsc-public-a --query
"Subnets[].[CidrBlock, AvailabilityZone]" --output text
aws ec2 describe-subnets --filters Name=tag:Name,Values=wsc-public-c --query
"Subnets[].[CidrBlock, AvailabilityZone]" --output text
aws ec2 describe-subnets --filters Name=tag:Name,Values=wsc-private-a --query
"Subnets[].[CidrBlock, AvailabilityZone]" --output text
aws ec2 describe-subnets --filters Name=tag:Name,Values=wsc-private-c --query
"Subnets[].[CidrBlock, AvailabilityZone]" --output text
aws ec2 describe-subnets --filters Name=tag:Name,Values=wsc-workload-a --query
"Subnets[].[CidrBlock, AvailabilityZone]" --output text
aws ec2 describe-subnets --filters Name=tag:Name,Values=wsc-workload-c --query
"Subnets[].[CidrBlock, AvailabilityZone]" --output text</td></tr>
<tr><td>1-1-B
(예상 출력)
<u>정확히 일치</u>
<u>순서 중요</u></td><td>10.0.0.0/24 ap-northeast-2a
10.0.1.0/24 ap-northeast-2c
10.0.2.0/24 ap-northeast-2a
10.0.3.0/24 ap-northeast-2c
10.0.4.0/24 ap-northeast-2a
10.0.5.0/24 ap-northeast-2c</td></tr>
</table>

순번	채점항목	
1-1	1-1-C (명령어 입력)	aws ec2 describe-route-tables --filter Name=tag:Name,Values=wsc-public-rtb --query "RouteTables[].Routes[]" | grep "igw-" | wc -l aws ec2 describe-route-tables --filter Name=tag:Name,Values=wsc-private-a-rtb --query "RouteTables[].Routes[].NatGatewayId" | grep -E "nat-|vpce-" | wc -l aws ec2 describe-route-tables --filter Name=tag:Name,Values=wsc-private-c-rtb --query "RouteTables[].Routes[].NatGatewayId" | grep -E "nat-|vpce-" | wc -l aws ec2 describe-route-tables --filter Name=tag:Name,Values=wsc-workload-a-rtb --query "RouteTables[].Routes[]" | grep -E "igw-|nat-|vpce-" | wc -l aws ec2 describe-route-tables --filter Name=tag:Name,Values=wsc-workload-c-rtb --query "RouteTables[].Routes[]" | grep -E "igw-|nat-|vpce-" | wc -l
	1-1-C (예상 출력) **정확히 일치** **순서 중요**	1 1 1 0 0
2-1	2-1-A (명령어 입력)	aws dynamodb describe-table --table-name wsc-table --query "Table.AttributeDefinitions" --output text
	2-1-A (예상 출력) **정확히 일치**	client_id S
	2-1-B (명령어 입력)	aws dynamodb describe-table --table-name wsc-table --query "Table.SSEDescription.KMSMasterKeyArn" --output text
	2-1-B (예상 출력)	“arn:aws:kms “로 시작하는 문자열
3-1	3-1-A (명령어 입력)	ACCOUNT_ID=$(aws sts get-caller-identity --query "Account" --output text) aws s3api list-objects-v2 --bucket wsc-static-$ACCOUNT_ID --query "Contents[].Key"
	3-1-A (예상 출력) **정확히 일치**	["static/index.html", "static/main.jpeg"]
	3-1-B (명령어 입력)	ACCOUNT_ID=$(aws sts get-caller-identity --query "Account" --output text) aws s3api get-bucket-encryption --bucket wsc-static-$ACCOUNT_ID --query "ServerSideEncryptionConfiguration.Rules[].ApplyServerSideEncryptionByDefault.KMSMasterKeyID" --output text aws s3api head-object --bucket wsc-static-$ACCOUNT_ID --key static/index.html --query "SSEKMSKeyId" --output text aws s3api head-object --bucket wsc-static-$ACCOUNT_ID --key static/main.jpeg --query "SSEKMSKeyId" --output text
	3-1-B (예상 출력)	“arn:aws:kms “로 시작하는 문자열 “arn:aws:kms “로 시작하는 문자열 “arn:aws:kms “로 시작하는 문자열
4-1	4-1-A (명령어 입력)	aws lambda get-function-configuration --function-name wsc-get-table-function --query "Runtime" --output text
	4-1-A (예상 출력) **정확히 일치**	python3.14

순번	채점항목	
4-1	4-1-B (명령어 입력)	LAMBDA_VPC_SUBNETS=$(aws lambda get-function-configuration --function-name wsc-get-table-function --query "VpcConfig.SubnetIds" --output text) aws ec2 describe-subnets --subnet-ids $LAMBDA_VPC_SUBNETS --query "Subnets[*].MapPublicIpOnLaunch" --output text
	4-1-B (예상 출력) **정확히 일치**	False False
5-1	5-1-A (명령어 입력)	aws ecr describe-repositories --repository-names wsc-repo --query "repositories[].{imageTagMutability:imageTagMutability,scanOnPush:imageScanningConfiguration.scanOnPush,encryptionConfiguration:encryptionConfiguration.encryptionType}"

	5-1-A
	(예상 출력)
	정확히 일치
		[
		  {
		  "imageTagMutability": "MUTABLE",
		  "scanOnPush": true,
		  "encryptionConfiguration": "KMS"
		  }
		]

5-2	5-2-A (명령어 입력)	IMAGE_SIZE_BYTES=$(aws ecr describe-images --repository-name "wsc-repo" --query 'imageDetails[?imageTags[0]==`v1.0.0`].imageSizeInBytes' --output text) IMAGE_SIZE_MB=$(awk "BEGIN {printf W"%.2fW", $IMAGE_SIZE_BYTES / 1024 / 1024}") echo "wsc-repo: ${IMAGE_SIZE_MB}mb"
	5-2-A (예상 출력)	**# 출력된 값이 8MB 이하인지 확인** 5.05MB
	5-2-B (명령어 입력)	aws ecr describe-image-scan-findings --repository-name wsc-repo --image-id imageTag=v1.0.0 --query "imageScanStatus.status" --output text aws ecr describe-image-scan-findings --repository-name wsc-repo --image-id imageTag=v1.0.0 --query "imageScanFindings.findingSeverityCounts" --output text
	5-2-B (예상 출력) **정확히 일치**	COMPLETE
6-1	6-1-A (명령어 입력)	aws eks describe-cluster --name wsc-eks-cluster --query "cluster.version" --output text aws eks describe-cluster --name wsc-eks-cluster --query 'cluster.logging.clusterLogging[].types' --output text aws eks describe-cluster --name wsc-eks-cluster --query "cluster.resourcesVpcConfig.[endpointPublicAccess, endpointPrivateAccess]" --output text
	6-1-A (예상 출력) **정확히 일치**	1.35 api audit authenticator controllerManager scheduler False True
	6-1-B (명령어 입력)	aws eks describe-cluster --name wsc-eks-cluster --query "cluster.encryptionConfig[].provider.keyArn" --output text
	6-1-B (예상 출력)	“arn:aws:kms “로 시작하는 문자열

순번	채점항목	

6-2	6-2-A
	(명령어 입력)
		NODE_GROUP_LABELS=("type=app" "type=addon" "type=monitoring")
		
		for LABEL in "${NODE_GROUP_LABELS[@]}"; do
		  kubectl get no -l $LABEL -o json | jq -r
		'[.items[].metadata.labels["eks.amazonaws.com/nodegroup"]] | join(" ")'
		  kubectl get no -l $LABEL -o json | jq -r '.items[].metadata.name'
		  kubectl get no -l $LABEL -o json | jq -r
		'[.items[].metadata.labels["beta.kubernetes.io/instance-type"]] | join(" ")'
		  echo
		done

	6-2-A (예상 출력) **붉은색 부분 제외** **정확히 일치**	wsc-app-ng wsc-app-ng i-03920f8222ff0fc45.ap-northeast-2.compute.internal i-0bb6d1e5eeb88a5aa.ap-northeast-2.compute.internal t3.medium t3.medium wsc-addon-ng wsc-addon-ng i-0715e1602720db3b2.ap-northeast-2.compute.internal i-0ca1f4e7573a4d51d.ap-northeast-2.compute.internal t3.medium t3.medium wsc-monitoring-ng wsc-monitoring-ng i-087a479e25963dd01.ap-northeast-2.compute.internal i-0980ac3d349ed2b53.ap-northeast-2.compute.internal t3.medium t3.medium

6-3	6-3-A
	(명령어 입력)
		NODE_GROUP_NAMES=("wsc-app-node" "wsc-addon-node" "wsc-monitoring-node")
		for NODE_GROUP_NAME in "${NODE_GROUP_NAMES[@]}"; do
		  NODE_GROUP_INSTANCE_ID=$(aws ec2 describe-instances --filters
		"Name=tag:Name,Values=$NODE_GROUP_NAME" "Name=instance-state-name,Values=running" --query
		"Reservations[0].Instances[0].InstanceId" --output text)
		  aws ec2 describe-volumes --filters
		Name=attachment.instance-id,Values=$NODE_GROUP_INSTANCE_ID --query "Volumes[0].Encrypted"
		--output text
		done

	6-3-A (예상 출력) **정확히 일치**	True True True
6-4	6-4-A (명령어 입력)	sudo dnf install -y sshpass > /dev/null NODE_GROUP_PRIVATE_IP=$(kubectl get no -o json \| jq -r '.items[0].status.addresses[] \| select(.type=="InternalIP") \| .address') sshpass -p 'Skill53##' ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -o LogLevel=ERROR ec2-user@$NODE_GROUP_PRIVATE_IP "ping 1.1.1.1 -c 1" sshpass -p 'Skill53##' ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -o LogLevel=ERROR ec2-user@$NODE_GROUP_PRIVATE_IP "ping 8.8.8.8 -c 1" sshpass -p 'Skill53##' ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -o LogLevel=ERROR ec2-user@$NODE_GROUP_PRIVATE_IP "curl -sS -m 5 https://www.naver.com"
	6-4-A (예상 출력) **붉은색 부분** **정확히 일치**	PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data. --- 1.1.1.1 ping statistics --- 1 packets transmitted, 0 received, 100% packet loss, time 0ms PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data. --- 8.8.8.8 ping statistics --- 1 packets transmitted, 0 received, 100% packet loss, time 0ms curl: (28) Connection timed out after 5001 milliseconds

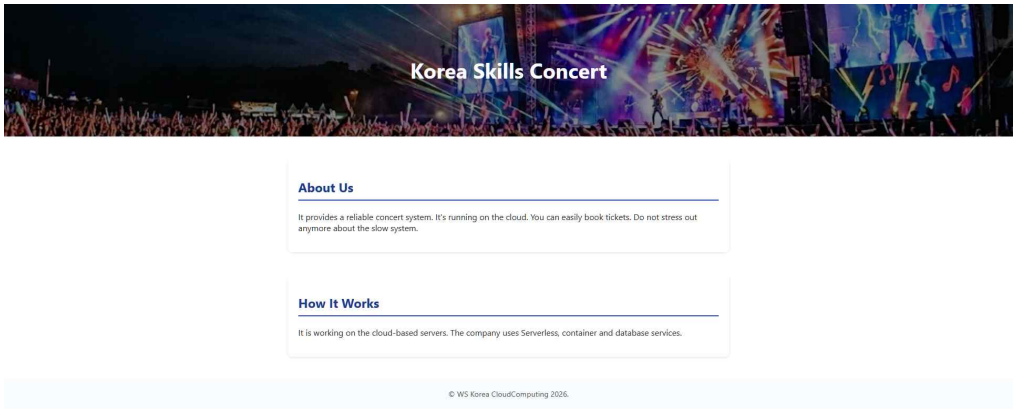
순번	채점항목	
6-5	6-5-A (명령어 입력)	kubectl exec deploy/wsc-deploy -n wsc -- curl -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600" --max-time 10 2>&1 | grep "Operation timed out"
	6-5-A (예상 출력)	**# 붉은색 부분 제외 정확히 일치** curl: (28) Operation timed out after 10002 milliseconds with 0 bytes received
6-6	6-6-A (명령어 입력)	kubectl -n kube-system get configmap coredns -o jsonpath='{.data.Corefile}' | grep -o 'kubernetes wsc.local'
	6-6-A (예상 출력) **정확히 일치**	kubernetes wsc.local
6-7	6-7-A (명령어 입력)	kubectl get deploy -n wsc -o custom-columns=NAME:.metadata.name,READY:.status.readyReplicas
	6-7-A (예상 출력) **정확히 일치**	NAME READY wsc-deploy 2
	6-7-B (명령어 입력)	kubectl get deploy wsc-deploy -n wsc -o json | jq -e '.spec.template.spec.containers[].env as $env \| any($env[]; .valueFrom.configMapKeyRef.name != null) and (all($env[]; has("value") | not))' >/dev/null && echo "PASS" || echo "FAIL"
	6-7-B (예상 출력) **정확히 일치**	PASS
	6-7-C (명령어 입력)	kubectl get sc wsc-sc -o jsonpath='{.parameters.encrypted}' kubectl get sc wsc-sc -o jsonpath='{.parameters.kmsKeyId}'
	6-7-C (예상 출력)	true “arn:aws:kms “로 시작하는 문자열
	6-7-D (명령어 입력)	kubectl get pvc -n monitoring -o custom-columns=NAME:.metadata.name,STATUS:.status.phase kubectl get pv -o custom-columns=NAME:.metadata.name,STATUS:.status.phase,CLAIM:.spec.claimRef.name,STORAGECLASS:.spec.storageClassName
	6-7-D (예상 출력) **정확히 일치**	STATUS CLAIM STORAGECLASS Bound wsc-prometheus-pvc wsc-sc Bound wsc-grafana-pvc wsc-sc
7-1	7-1-A (명령어 입력)	APP_LB_VPC_SUBNETS=$(aws elbv2 describe-load-balancers --name wsc-app-lb --query "LoadBalancers[].AvailabilityZones[].SubnetId" --output text) aws elbv2 describe-load-balancers --name wsc-app-lb --query "LoadBalancers[].Scheme" --output text aws elbv2 describe-load-balancers --name wsc-app-lb --query "LoadBalancers[].AvailabilityZones[].ZoneName" --output json | jq -r 'sort | join(" ")' aws ec2 describe-subnets --subnet-ids $APP_LB_VPC_SUBNETS --query "Subnets[*].MapPublicIpOnLaunch" --output text
	7-1-A (예상 출력) **정확히 일치**	internal ap-northeast-2a ap-northeast-2c False False
	7-1-B (명령어 입력)	APP_LB_DNS=$(aws elbv2 describe-load-balancers --name wsc-app-lb --query "LoadBalancers[].DNSName" --output text) curl -sS -X GET "http://$APP_LB_DNS/health" --max-time 10
	7-1-B (예상 출력)	**# 붉은색 부분 제외 정확히 일치** curl: (28) Connection timed out after 10002 milliseconds

순번	채점항목	
7-1	7-1-C (명령어 입력)	ADDON_LB_VPC_SUBNETS=$(aws elbv2 describe-load-balancers --name wsc-addon-lb --query "LoadBalancers[].AvailabilityZones[].SubnetId" --output text) aws elbv2 describe-load-balancers --name wsc-addon-lb --query "LoadBalancers[].Scheme" --output text aws elbv2 describe-load-balancers --name wsc-addon-lb --query "LoadBalancers[].AvailabilityZones[].ZoneName" --output json | jq -r 'sort | join(" ")' aws ec2 describe-subnets --subnet-ids $ADDON_LB_VPC_SUBNETS --query "Subnets[*].MapPublicIpOnLaunch" --output text
	7-1-C (예상 출력) **정확히 일치**	internet-facing ap-northeast-2a ap-northeast-2c True True
8-1	8-1-A (명령어 입력)	CDN_DOMAIN=$(aws resourcegroupstaggingapi get-resources --tag-filters Key=Name,Values=wsc-cdn --resource-type-filters 'cloudfront' --region us-east-1 --query "ResourceTagMappingList[0].ResourceARN" --output text \| sed 's:.*/::' \| xargs -I {} aws cloudfront get-distribution --id {} --query "Distribution.DomainName" --output text) aws cloudfront list-distributions --query "DistributionList.Items[?DomainName=='$CDN_DOMAIN'].PriceClass" --output text aws cloudfront list-distributions --query "DistributionList.Items[?DomainName=='$CDN_DOMAIN'].IsIPV6Enabled" --output text
	8-1-A (예상 출력) **정확히 일치**	PriceClass_All False
8-2	8-2-A (명령어 입력)	for i in {1..5}; do curl -s -I https://$CDN_DOMAIN/index.html | grep -i "x-cache" | grep -i "Hit from cloudfront" && break done
	8-2-A (예상 출력) **정확히 일치**	x-cache: Hit from cloudfront
	8-2-B (명령어 입력)	curl --silent -i -X GET --max-time 5 -w "₩n%{http_code}₩n" http://$CDN_DOMAIN/index.html \| grep -iE "x-cache:\|^301$"
	8-2-B (예상 출력) **정확히 일치**	X-Cache: Redirect from cloudfront 301
9-1	9-1-A (명령어 입력)	WAF_ID=$(aws wafv2 list-web-acls --scope CLOUDFRONT --region us-east-1 --query "WebACLs[?Name=='wsc-waf'].Id" --output text) aws wafv2 get-web-acl --scope CLOUDFRONT --region us-east-1 --name wsc-waf --id "$WAF_ID" | jq -r '.WebACL.Rules[] | .. | .SearchString? // empty' | while read -r s; do echo "$s" | base64 --decode; echo; done | grep -E "admin|sysop" | sort -u
	9-1-A (예상 출력) **정확히 일치**	admin sysop
10-1	10-1-A (명령어 입력)	CLIENT_ID="C$((RANDOM % 900 + 100))" USERNAME=$(tr -dc 'A-Za-z0-9' </dev/urandom | head -c 8) EMAIL="$USERNAME@example.com" CONCERT_NAME=$(tr -dc 'A-Za-z0-9' </dev/urandom | head -c 10) curl -sS -w "%{http_code}₩n" -X POST "https://$CDN_DOMAIN/v1/book" -H "Content-Type: application/json" -d "{₩"client_id₩": ₩"$CLIENT_ID₩", ₩"username₩": ₩"$USERNAME₩", ₩"email₩": ₩"$EMAIL₩", ₩"concert_name₩": ₩"$CONCERT_NAME₩"}"
	10-1-A (예상 출력)	**# 붉은색 부분 정확히 일치** {"booking_id":"WUYUYQ3C"} 200

순번	채점항목	
10-2	10-2-A (명령어 입력)	curl -sS -w "Wn%{http_code}Wn" -X GET "https://$CDN_DOMAIN/v1/book?client_id=$CLIENT_ID" | jq

	10-2-A
	(예상 출력)
		# 붉은색 부분 정확히 일치
		{
		"client_id": "C783",
		"booking_id": "WUYUYQ3C",
		"username": "Hg2KwehC",
		"email": "Hg2KwehC@example.com",
		"concert_name": "konYZjq8WC"
		}
		200

10-3	10-3-A (명령어 입력)	APP_LB_DNS=$(aws elbv2 describe-load-balancers --name wsc-app-lb --query "LoadBalancers[].DNSName" --output text) curl -sS -X GET "http://$APP_LB_DNS/health" --max-time 10
	10-3-A (예상 출력)	**# 붉은색 부분 제외 정확히 일치** curl: (28) Connection timed out after 10002 milliseconds
	10-3-B (명령어 입력)	curl -sS -o /dev/null -w "%{http_code}Wn" -X POST "https://$CDN_DOMAIN/v1/book" -H "Content-Type: application/json" -d "{W"client_idW": W"$CLIENT_IDW", W"usernameW": W"AdminW", W"emailW": W"$EMAILW", W"concert_nameW": W"$CONCERT_NAME W"}" curl -sS -o /dev/null -w "%{http_code}Wn" -X POST "https://$CDN_DOMAIN/v1/book" -H "Content-Type: application/json" -d "{W"client_idW": W"$CLIENT_IDW", W"usernameW": W"SysopW", W"emailW": W"$EMAILW", W"concert_nameW": W"$CONCERT_NAME W"}" curl -sS -o /dev/null -w "%{http_code}Wn" -X POST "https://$CDN_DOMAIN/v1/book" -H "Content-Type: application/json" -d "{W"client_idW": W"$CLIENT_IDW", W"usernameW": W"$USERNAMEW", W"emailW": W"admin@example.comW", W"concert_nameW": W"$CONCERT_NAME W"}" curl -sS -o /dev/null -w "%{http_code}Wn" -X POST "https://$CDN_DOMAIN/v1/book" -H "Content-Type: application/json" -d "{W"client_idW": W"$CLIENT_IDW", W"usernameW": W"$USERNAMEW", W"emailW": W"sysop@example.comW", W"concert_nameW": W"$CONCERT_NAME W"}"
	10-3-B (예상 출력) **정확히 일치**	403 403 403 403

<table>
<tr><th>순번</th><th colspan="2">채점항목</th></tr>
<tr><td rowspan="2">10-4</td><td>10-4-A
(콘솔 접속)</td><td>1. 터미널에 출력된 URL을 복사한 후 Firefox 또는 Chrome 등과 같은 Web Browser에 붙여넣어 접근합니다.</td></tr>
<tr><td>10-5-A
(예상 출력)
<u>정확히 일치</u></td><td> 
</td></tr>
<tr><td rowspan="4">11-1</td><td>11-1-A
(명령어 입력)</td><td>ADDON_LB_DNS=$(aws elbv2 describe-load-balancers --name wsc-addon-lb --query "LoadBalancers[].DNSName" --output text)

curl -s http://$ADDON_LB_DNS/prometheus/-/healthy | grep -q "Prometheus Server is Healthy." && echo "up" || echo "down"
curl -s -G http://$ADDON_LB_DNS/prometheus/api/v1/query --data-urlencode 'query=min(up)' | jq -r '.data.result[0].value[1]'</td></tr>
<tr><td>11-1-A
(예상 출력)
<u>정확히 일치</u></td><td>up
1</td></tr>
<tr><td>11-1-B
(명령어 입력)</td><td>ADDON_LB_DNS=$(aws elbv2 describe-load-balancers --name wsc-addon-lb --query "LoadBalancers[].DNSName" --output text)
GRAFANA_DASHBOARD_UID=$(curl -s -u admin:'Skill53##' http://$ADDON_LB_DNS/grafana/api/search | jq -r '.[] | select(.title=="wsc-eks-dashboard") | .uid')
DASH=$(curl -s -u admin:'Skill53##' http://$ADDON_LB_DNS/grafana/api/dashboards/uid/$GRAFANA_DASHBOARD_UID)

curl -sf http://$ADDON_LB_DNS/grafana/api/health | jq -r '.database' | grep -q "ok" && echo "up" || echo "down"
curl -s -u admin:'Skill53##' http://$ADDON_LB_DNS/grafana/api/search | jq -r '.[] | select(.title=="wsc-eks-dashboard") | .title'
curl -s -u admin:'Skill53##' http://$ADDON_LB_DNS/grafana/api/datasources | jq -r '.[] | select(.name=="Prometheus") | .type'
curl -s -u admin:'Skill53##' http://$ADDON_LB_DNS/grafana/api/datasources | jq -r '.[] | select(.name=="Prometheus") | .url'
echo "$DASH" | jq '.dashboard.panels | length'</td></tr>
<tr><td>11-1-B
(예상 출력)
<u>정확히 일치</u></td><td>up
wsc-eks-dashboard
prometheus
http://prometheus-server.monitoring.svc.wsc.local/prometheus
6</td></tr>
</table>

순번	채점항목	
11-2	11-2-A (명령어 입력)	echo "$DASH" \| jq '.dashboard.panels[] \| select(.title=="TOTAL_NODE_GROUP_COUNT") \| .type' EXPR=$(echo "$DASH" \| jq -r '.dashboard.panels[] \| select(.title=="TOTAL_NODE_GROUP_COUNT") \| .targets[].expr') curl -s -G http://$ADDON_LB_DNS/prometheus/api/v1/query --data-urlencode "query=$EXPR" | jq -r '.data.result[0].value[1]'
	11-2-A (예상 출력) **정확히 일치**	"stat" 6
	11-2-B (명령어 입력)	echo "$DASH" \| jq '.dashboard.panels[] \| select(.title=="APP_POD_COUNT") \| .type' EXPR=$(echo "$DASH" \| jq -r '.dashboard.panels[] \| select(.title=="APP_POD_COUNT") \| .targets[].expr') curl -s -G http://$ADDON_LB_DNS/prometheus/api/v1/query --data-urlencode "query=$EXPR" | jq -r '.data.result[0].value[1]'
	11-2-B (예상 출력) **정확히 일치**	"stat" 2
	11-2-C (명령어 입력)	echo "$DASH" \| jq '.dashboard.panels[] \| select(.title=="NODE_GROUP_CPU_USAGE") \| .type' EXPR=$(echo "$DASH" \| jq -r '.dashboard.panels[] \| select(.title=="NODE_GROUP_CPU_USAGE") \| .targets[].expr') curl -s -G http://$ADDON_LB_DNS/prometheus/api/v1/query --data-urlencode "query=$EXPR" \| jq '.data.result \| length' echo "$EXPR" | grep -q "cpu" && echo "CPU"
	11-2-C (예상 출력) **정확히 일치**	"timeseries" 6 CPU
	11-2-D (명령어 입력)	echo "$DASH" \| jq '.dashboard.panels[] \| select(.title=="NODE_GROUP_MEMORY_USAGE") \| .type' EXPR=$(echo "$DASH" \| jq -r '.dashboard.panels[] \| select(.title=="NODE_GROUP_MEMORY_USAGE") \| .targets[].expr') curl -s -G http://$ADDON_LB_DNS/prometheus/api/v1/query --data-urlencode "query=$EXPR" \| jq '.data.result \| length' echo "$EXPR" | grep -q "memory" && echo "Memory"
	11-2-D (예상 출력) **정확히 일치**	"timeseries" 6 Memory
	11-2-E (명령어 입력)	echo "$DASH" \| jq '.dashboard.panels[] \| select(.title=="APP_POD_CPU_USAGE") \| .type' EXPR=$(echo "$DASH" \| jq -r '.dashboard.panels[] \| select(.title=="APP_POD_CPU_USAGE") \| .targets[].expr') curl -s -G http://$ADDON_LB_DNS/prometheus/api/v1/query --data-urlencode "query=$EXPR" \| jq '.data.result \| length' echo "$EXPR" | grep -q "cpu" && echo "CPU"
	11-2-E (예상 출력) **정확히 일치**	"bargauge" 2 CPU
	11-2-F (명렁어 입력) **정확히 일치**	echo "$DASH" \| jq '.dashboard.panels[] \| select(.title=="APP_POD_MEMORY_USAGE") \| .type' EXPR=$(echo "$DASH" \| jq -r '.dashboard.panels[] \| select(.title=="APP_POD_MEMORY_USAGE") \| .targets[].expr') curl -s -G http://$ADDON_LB_DNS/prometheus/api/v1/query --data-urlencode "query=$EXPR" \| jq '.data.result \| length' echo "$EXPR" | grep -q "memory" && echo "Memory"
	11-2-F (예상 출력) **정확히 일치**	"bargauge" 2 Memory

순번	채점항목	
12-1	12-1-A (명령어 입력)	aws logs describe-log-groups --log-group-name-prefix /wsc/pod/log --query "logGroups[].kmsKeyId" --output text
	12-1-A (예상 출력)	“arn:aws:kms “로 시작하는 문자열
	12-1-B (명령어 입력)	APP_LB_DNS=$(aws elbv2 describe-load-balancers --name wsc-app-lb --query "LoadBalancers[].DNSName" --output text) curl -s -o /dev/null -X GET "http://$APP_LB_DNS/health" --max-time 10 sleep 1m aws logs filter-log-events --log-group-name /wsc/pod/log --filter-pattern '"/health"' | jq ".events | length"
	12-1-B (예상 출력) **정확히 일치**	0
	12-1-C (명령어 입력)	CLIENT_ID="C$((RANDOM % 900 + 100))" USERNAME=$(tr -dc 'A-Za-z0-9' </dev/urandom | head -c 8) EMAIL="$USERNAME@example.com" CONCERT_NAME=$(tr -dc 'A-Za-z0-9' </dev/urandom | head -c 10) aws logs describe-log-streams --log-group-name /wsc/pod/log --query "logStreams[].logStreamName" --output text | tr '₩t' '₩n' | while read stream; do aws logs delete-log-stream --log-group-name /wsc/pod/log --log-stream-name "$stream" done kubectl rollout restart daemonset fluent-bit -n logging > /dev/null sleep 15 curl -sS -o /dev/null -X POST "https://$CDN_DOMAIN/v1/book" -H "Content-Type: application/json" -d "{₩"client_id₩": ₩"$CLIENT_ID₩", ₩"username₩": ₩"$USERNAME₩", ₩"email₩": ₩"$EMAIL₩", ₩"concert_name₩": ₩"$CONCERT_NAME₩"}" sleep 1m aws logs filter-log-events --log-group-name /wsc/pod/log --filter-pattern '"/v1/book"' | jq ".events | length" echo
	12-1-C (예상 출력) **정확히 일치**	1